



**UNIVERSIDAD PRIVADA DEL VALLE FACULTAD
DE INFORMÁTICA Y ELECTRÓNICA
CARRERA DE LICENCIATURA INGENIERÍA DE
SISTEMAS INFORMÁTICOS**

PROGRAMACIÓN WEB II

**MANUAL TECNICO DE SISTEMA DE SASTRERIA
“PONCHOS BLANCOS”**

Paralelo: “A”

Estudiantes:

- **Yoann Sebastian Cachi Felipe**
- **José Gabriel Aroyo Santos**
- **Dustin Fernando Vaca Aguilar**

Docente: Ing. Henry Miranda.

La Paz - Bolivia

Gestión I – 2026

INDICE DE CONTENIDO

1	INTRODUCCIÓN	1
2	DESCRIPCIÓN GENERAL DEL SISTEMA.....	1
3	OBJETIVOS DEL PROYECTO	2
3.1	Objetivo General:.....	2
3.2	Objetivos Específicos:.....	2
4	ALCANCE Y LIMITACIONES	3
5	ARQUITECTURA DEL SISTEMA	4
5.1	CAPA DE PRESENTACIÓN	4
5.2	CAPA DE NEGOCIO	5
5.3	CAPA DE DATOS.....	6
6	TECNOLOGÍAS UTILIZADAS	7
7	ESTRUCTURA DEL PROYECTO.....	9
8	DISEÑO DE BASE DE DATOS	10
9	MÓDULOS DEL SISTEMA	10
10	COMUNICACIÓN FRONTEND-BACKEND	11
11	VALIDACIONES IMPLEMENTADAS	12
11.1	Validaciones de Empleados.....	12
11.2	Validaciones de Clientes.....	13

11.3	Validaciones de Pedidos.....	13
11.4	Validaciones de Pagos	13
11.5	Validaciones de Entregas	14
12	SEGURIDAD.....	14
13	FLUJO COMPLETO DEL SISTEMA	15
14	INSTALACIÓN Y CONFIGURACIÓN	16
15	PRUEBAS DEL SISTEMA.....	17
15.1	Pruebas CRUD	17
15.2	Pruebas autenticación	18
15.3	Pruebas validación	18
15.4	Pruebas persistencia	18
16	MANTENIMIENTO	18
17	POSIBLES MEJORAS	19
18	CONCLUSIONES	20

1 INTRODUCCIÓN

El Sistema Web de Gestión de Sastrería fue desarrollado con el objetivo de automatizar procesos administrativos y operativos relacionados con la administración interna de una sastrería. En muchas pequeñas empresas este tipo de actividades suelen realizarse utilizando registros manuales, hojas de cálculo o múltiples herramientas separadas, generando problemas de duplicidad de datos, pérdida de información, errores humanos y baja velocidad de atención.

Para resolver estas dificultades se diseñó una plataforma centralizada que permite gestionar información relacionada con empleados, clientes, pedidos, pagos y entregas desde una única interfaz web.

El sistema fue construido utilizando una arquitectura moderna basada en servicios API REST, permitiendo desacoplar completamente el frontend del backend y facilitando mantenimiento futuro.

Esta separación también permite que el sistema pueda evolucionar posteriormente hacia aplicaciones móviles o nuevas interfaces sin necesidad de reconstruir toda la lógica.

2 DESCRIPCIÓN GENERAL DEL SISTEMA

El sistema consiste en una aplicación web que funciona mediante autenticación y módulos independientes.

Cada módulo representa una entidad importante del negocio.

El usuario primero inicia sesión y posteriormente accede al panel principal.

Desde ese panel puede navegar hacia:

- Gestión de empleados.
- Gestión de clientes.
- Gestión de pedidos.
- Gestión de pagos.

- Gestión de entregas.

Cada módulo permite operaciones CRUD.

CRUD significa:

- Create => Crear.
- Read => Consultar.
- Update => Modificar.
- Delete => Eliminar.

La información registrada queda almacenada permanentemente dentro de la base de datos.

Toda la comunicación ocurre mediante solicitudes HTTP y respuestas JSON.

3 OBJETIVOS DEL PROYECTO

3.1 Objetivo General:

Diseñar e implementar un sistema web que permita administrar procesos internos de una sastrería de manera organizada y segura.

3.2 Objetivos Específicos:

- Automatizar el registro de empleados.
- Centralizar información.
- Reducir errores humanos.
- Controlar pedidos.
- Administrar pagos.
- Administrar entregas.
- Permitir crecimiento del sistema.

- Separar responsabilidades técnicas.

Estos objetivos guiaron todas las decisiones de diseño del proyecto.

4 ALCANCE Y LIMITACIONES

El sistema permite:

- Registro de usuarios.
- Autenticación.
- Administración de empleados.
- Administración de clientes.
- Administración de pedidos.
- Administración de pagos.
- Administración de entregas.
- Edición.
- Eliminación.
- Consulta.

Sin embargo, existen funcionalidades que quedaron fuera del alcance inicial.

Entre ellas:

- Generación automática de reportes.
- Facturación electrónica.
- Control de inventario.
- Envío de correos.

- Aplicación móvil.

Estas características podrían incorporarse en futuras versiones.

5 ARQUITECTURA DEL SISTEMA

Para el desarrollo del Sistema Web de Gestión de Sastrería se decidió implementar una arquitectura de tres capas. Esta arquitectura fue elegida porque permite dividir el sistema en componentes independientes, donde cada componente cumple responsabilidades específicas sin interferir directamente con el funcionamiento de los demás.

La principal ventaja de esta estructura es que mejora el mantenimiento del proyecto. Por ejemplo, si en el futuro se desea cambiar completamente el diseño visual del sistema, esto podría hacerse modificando únicamente el frontend sin afectar el procesamiento interno ni la base de datos. Del mismo modo, si se requiere cambiar el motor de base de datos o modificar reglas del negocio, estas modificaciones pueden realizarse sin rehacer todo el proyecto.

Además, esta arquitectura reduce el acoplamiento entre componentes, facilita detectar errores y permite reutilizar partes del sistema.

5.1 CAPA DE PRESENTACIÓN

La capa de presentación corresponde al frontend del sistema y representa la parte con la que interactúan directamente los usuarios.

Su objetivo principal es permitir que el usuario pueda visualizar información y ejecutar acciones mediante una interfaz gráfica amigable. Esta capa se encarga de mostrar formularios, botones, tablas, mensajes y elementos visuales que facilitan el uso del sistema.

Dentro de este proyecto, el frontend fue desarrollado utilizando HTML, CSS y JavaScript, mientras que Bootstrap fue utilizado como apoyo para construir interfaces visuales más ordenadas y adaptables.

HTML fue utilizado para estructurar el contenido de cada página. Cada pantalla del sistema fue construida utilizando etiquetas semánticas para mejorar organización y lectura del código.

CSS fue utilizado para personalizar la apariencia del sistema. Se definieron estilos propios dentro del archivo estilos.css para mantener una apariencia uniforme entre todas las pantallas.

JavaScript se utilizó para controlar la interacción con el usuario y realizar comunicación dinámica con el backend mediante solicitudes HTTP.

Bootstrap permitió reducir tiempo de desarrollo visual y asegurar que el sistema funcione correctamente en diferentes tamaños de pantalla.

Es importante destacar que esta capa no almacena información permanente ni realiza operaciones directamente sobre la base de datos. Todas las acciones realizadas por el usuario se transforman en solicitudes hacia el servidor.

5.2 CAPA DE NEGOCIO

La capa de negocio corresponde al backend del sistema.

Esta capa representa el núcleo del proyecto porque aquí se encuentra toda la lógica que define cómo debe comportarse el sistema.

Su función principal consiste en recibir solicitudes provenientes del frontend, validar información, aplicar reglas de negocio y devolver resultados.

Dentro del proyecto esta capa fue desarrollada utilizando Laravel.

Laravel fue seleccionado debido a que proporciona herramientas integradas para:

- Gestión de rutas.
- Controladores.
- Modelos.
- Validaciones.
- Autenticación.
- Seguridad.

- Respuestas API.

Una de las ventajas principales obtenidas fue poder trabajar utilizando controladores independientes para cada módulo.

Por ejemplo:

- EmployeeController administra empleados.
- CustomerController administra clientes.
- OrderController administra pedidos.
- PaymentController administra pagos.
- DeliveryController administra entregas.

Esto evita concentrar demasiada lógica en un solo archivo.

Además, Laravel permitió utilizar Eloquent ORM para trabajar con objetos en lugar de escribir consultas SQL manuales.

Ejemplo:

En lugar de escribir:

```
SELECT * FROM employees
```

Se utilizó:

```
Employee::all()
```

Esto mejora legibilidad y mantenimiento.

5.3 CAPA DE DATOS

La capa de datos corresponde al almacenamiento permanente del sistema.

Su responsabilidad consiste en guardar toda la información generada durante el uso del sistema.

Para este proyecto se utilizó MySQL debido a su estabilidad, facilidad de integración con Laravel y soporte para relaciones entre tablas.

La base de datos fue diseñada siguiendo principios de normalización.

Esto significa que la información se distribuyó entre tablas independientes evitando duplicaciones innecesarias.

Por ejemplo:

Los pedidos no almacenan nuevamente todos los datos del cliente.

En lugar de eso:

- orders
- customer_id
- customers

Esta estrategia reduce errores y facilita actualizaciones.

Además, la base de datos mantiene integridad mediante llaves primarias y foráneas, esto evita registros inconsistentes.

6 TECNOLOGÍAS UTILIZADAS

Las tecnologías seleccionadas fueron elegidas considerando facilidad de mantenimiento, compatibilidad entre herramientas y velocidad de desarrollo.

Laravel

Laravel fue utilizado como framework principal del backend, se eligió porque proporciona una estructura organizada que evita escribir código repetitivo.

Entre sus principales aportes al proyecto se encuentran:

- Sistema de rutas.

- Controladores.
- ORM.
- Migraciones.
- Validaciones.
- Autenticación.
- Gestión de errores.

Esto permitió construir una API estructurada y escalable.

HTML

HTML fue utilizado para construir la estructura visual del sistema, cada página representa una sección funcional independiente.

Por ejemplo:

- dashboard.html
- clientes.html
- empleados.html

HTML define formularios, tablas y navegación.

CSS

CSS fue utilizado para controlar completamente el diseño visual, se centralizó dentro del archivo estilos.css, esto permitió mantener coherencia visual entre todas las páginas, se trabajó utilizando clases reutilizables para reducir código repetido.

JavaScript

JavaScript fue utilizado para agregar comportamiento dinámico.

Su principal función fue consumir la API mediante fetch().

También permitió:

- Actualizar tablas.
- Mostrar alertas.
- Enviar formularios.
- Editar registros.
- Eliminar registros.
- **Sin recargar pantalla.**

Bootstrap

Bootstrap se utilizó para acelerar el diseño visual.

Sus componentes facilitaron:

- Tablas.
- Botones.
- Espaciado.
- Responsive.

MySQL

MySQL fue seleccionado como motor de almacenamiento, permite administrar relaciones entre tablas y garantizar persistencia de información incluso después de cerrar el sistema.

7 ESTRUCTURA DEL PROYECTO

La estructura del proyecto fue diseñada buscando organización y facilidad de mantenimiento, cada carpeta tiene una responsabilidad específica.

Esto permite localizar rápidamente archivos cuando sea necesario modificar el sistema.

Por ejemplo:

- `public/pages/`=Contiene únicamente interfaces.
- `public/js/`=Contiene comportamiento.
- `public/css/`=Contiene apariencia.
- `app/Http/Controllers/`=Contiene procesamiento.

Esta separación evita mezclar responsabilidades, además, mejora el trabajo colaborativo cuando varias personas participan simultáneamente.

8 DISEÑO DE BASE DE DATOS

La base de datos fue diseñada para representar correctamente las entidades reales de una sastrería.

Cada tabla representa un concepto del negocio.

- `users`: Controla autenticación.
- `employees`: Controla personal.
- `customers`: Controla clientes.
- `orders`: Controla pedidos.
- `payments`: Controla pagos.
- `deliveries`: Controla entregas.

Las relaciones fueron implementadas para evitar almacenar información repetida.

Esto mejora rendimiento y consistencia.

9 MÓDULOS DEL SISTEMA

Cada módulo fue diseñado para resolver una necesidad específica.

- Empleados: Administrar personal.
- Clientes: Administrar compradores.
- Pedidos: Controlar producción.
- Pagos: Registrar ingresos.
- Entregas: Controlar distribución.

Separar módulos mejora organización y crecimiento futuro.

10 COMUNICACIÓN FRONTEND-BACKEND

La comunicación fue implementada mediante API REST, el frontend nunca interactúa directamente con MySQL, esto mejora seguridad y evita modificaciones no autorizadas.

Cada solicitud sigue el flujo:

- Interfaz
- JavaScript
- Fetch
- Laravel
- Validación
- Base de datos
- Respuesta JSON
- Interfaz

Este diseño desacoplado facilita futuras integraciones.

11 VALIDACIONES IMPLEMENTADAS

Las validaciones representan uno de los componentes más importantes del sistema porque garantizan que únicamente se almacene información correcta y coherente dentro de la base de datos.

Durante el desarrollo se decidió implementar validaciones tanto en el frontend como en el backend. Esta decisión fue tomada debido a que validar únicamente desde la interfaz visual no proporciona seguridad suficiente, ya que cualquier usuario podría modificar solicitudes manualmente utilizando herramientas externas.

Las validaciones del frontend tienen como objetivo mejorar la experiencia del usuario evitando errores antes de enviar información al servidor.

Por otro lado, las validaciones del backend funcionan como una segunda capa obligatoria de verificación y son las responsables de garantizar integridad.

El sistema implementa validaciones para distintos tipos de datos.

11.1 Validaciones de Empleados

En el módulo de empleados se controlan datos relacionados con información laboral.

Se aplicaron restricciones para evitar registros imposibles o inconsistentes.

Ejemplos:

- Nombre: No puede estar vacío.
- Apellido: Debe contener texto válido.
- Teléfono: Debe cumplir formato esperado.
- Fecha de contratación: Debe corresponder a una fecha válida.
- Salario: No permite números negativos.
- Correo: Debe ser único.

- Contraseña: Debe coincidir con confirmación.

Estas reglas permiten mantener consistencia en los registros del personal.

11.2 Validaciones de Clientes

Los clientes representan una entidad fundamental porque participan directamente en pedidos.

Se validaron:

- Nombre.
- Apellido.
- Teléfono.
- Dirección.

Esto evita registros incompletos que posteriormente dificultarían entregas o pagos.

11.3 Validaciones de Pedidos

Los pedidos son uno de los procesos centrales del sistema.

Se controló:

- Fechas válidas.
- Precios positivos.
- Existencia de cliente.
- Estados válidos.

Esto evita pedidos imposibles de procesar.

11.4 Validaciones de Pagos

Se verificó:

- Montos positivos.
- Formato de fecha.
- Existencia del pedido asociado.

Esto asegura que no existan movimientos financieros inválidos.

11.5 Validaciones de Entregas

Se controló:

- Direcciones.
- Teléfonos.
- Fechas.
- Pedidos relacionados.

Estas restricciones ayudan a evitar errores logísticos.

12 SEGURIDAD

La seguridad fue considerada desde el inicio del desarrollo debido a que el sistema almacena información sensible relacionada con empleados y operaciones internas.

Para proteger la información se implementó un sistema de autenticación basado en tokens utilizando Laravel Sanctum.

El objetivo principal fue garantizar que únicamente usuarios autenticados puedan interactuar con el sistema.

El funcionamiento es el siguiente:

- Usuario ingresa credenciales.
- Backend verifica.
- Genera token.

- Frontend almacena.
- Cada solicitud envía token.
- Servidor valida.
- Permite acceso.

Si el token no existe o es inválido, el sistema rechaza automáticamente la solicitud.

13 FLUJO COMPLETO DEL SISTEMA

El funcionamiento completo del sistema sigue una secuencia definida.

Paso 1.

Ingreso al sistema: El usuario abre la aplicación.

Paso 2.

Autenticación: Se validan credenciales.

Paso 3.

Generación de sesión: Se almacena token.

Paso 4.

Carga del Dashboard: Se muestran módulos.

Paso 5.

Selección del módulo: Usuario ingresa.

Paso 6.

Operación:

- Crear.

- Consultar.
- Editar.
- Eliminar.

Paso 7.

Procesamiento: Laravel ejecuta.

Paso 8.

Persistencia: Se guarda.

Paso 9.

Respuesta: Actualización visual.

Este flujo garantiza orden y trazabilidad.

14 INSTALACIÓN Y CONFIGURACIÓN

El sistema fue diseñado para ser desplegado localmente utilizando Laravel.

Proceso completo:

Instalar dependencias:

composer Install: Esto descarga paquetes.

Configurar entorno:

Editar .env

Configurar:

- APP_URL
- DB_DATABASE

- DB_USERNAME
- DB_PASSWORD

Migrar:

php artisan migrate: Esto crea tablas.

Ejecutar servidor:

php artisan serve

Acceder:

http://127.0.0.1:8000

Verificar funcionamiento:

Login.

Dashboard.

CRUD.

15 PRUEBAS DEL SISTEMA

Las pruebas fueron necesarias para garantizar estabilidad.

Se realizaron pruebas manuales y funcionales.

15.1 Pruebas CRUD

- Crear.
- Editar.
- Eliminar.
- Consultar.

15.2 Pruebas autenticación

- Login correcto.
- Login incorrecto.
- Registro.
- Sesión.

15.3 Pruebas validación

- Campos vacíos.
- Datos negativos.
- Formatos inválidos.
- Pruebas navegación
- Carga.
- Actualización.
- Rutas.

15.4 Pruebas persistencia

- Guardar.
- Modificar.
- Eliminar.

16 MANTENIMIENTO

El mantenimiento fue considerado para facilitar futuras modificaciones.

Se recomienda:

Limpiar caché:

php artisan optimize:clear

Actualizar:

composer update

Ver rutas:

php artisan route:list

Migrar nuevamente:

php artisan migrate

17 POSIBLES MEJORAS

Aunque el sistema cumple los objetivos actuales, existen oportunidades de expansión.

Posibles mejoras:

- Sistema de reportes.
- Panel estadístico.
- Inventario.
- Control de roles.
- Subida imágenes.
- Exportación Excel.
- Notificaciones.
- Historial cambios.
- Aplicación móvil.

- API pública.

Estas mejoras permitirían convertir el sistema en una plataforma más completa.

18 CONCLUSIONES

El Sistema Web de Gestión de Sastrería fue desarrollado aplicando principios de organización, separación de responsabilidades y buenas prácticas de desarrollo.

La arquitectura implementada permitió mantener independencia entre interfaz, procesamiento y almacenamiento.

Las validaciones incorporadas reducen errores y mejoran calidad de datos.

La autenticación implementada protege la información almacenada.

Finalmente, el sistema proporciona una solución funcional para administrar procesos internos de una sastrería y deja una base preparada para futuras ampliaciones tecnológicas.